KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 02

Laboratuvar Föyü

Yapıcı metotlar · this · Aşırı yükleme · Kapsüllemeye ilk adım

ÖZET

Bu laboratuvar oturumunda geçen hafta yazdığınız `Kitap` sınıfını yapıcı metotlarla yeniden yazacak, aşırı yüklenmiş yapıcılar ve `this(...)` zinciri kuracak, `this` ile alan gölgelemesini çözecek, yapıcı hatalarıyla dolu bir kodu düzeltecek ve son olarak kapsülleme ile kaynak yönetimini birlikte kullanan bir banka hesabı sınıfı yazacaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 2 ders notu Bölüm 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta02_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- Bir sınıfa tam yapıcı yazıp nesneyi doğduğu anda geçerli duruma getirebilir.
- Doğrulama mantığını yapıcının ve setter'ın içine yerleştirebilir.
- Aşırı yüklenmiş yapıcılar yazabilir ve `this(...)` ile zincirleyebilir.
- `this` anahtar sözcüğü ile alan gölgelemesini çözebilir.
- `return this;` ile zincirleme çağrı yazabilir.
- Kopya yapıcı yazabilir ve `==` ile bağımsızlığı sınayabilir.
- `toString()` metodunu ezerek biçimli metin gösterimi üretebilir.
- Yapıcı ve `this` ile ilgili derleyici hatalarını okuyup düzeltebilir.



Şekil 2.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

- Her sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.
- Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`

- Sınıf adları `PascalCase`, değişken ve metot adları `camelCase` olmalıdır.
- Değişken, sınıf ve metot adlarında **Türkçe karakter kullanmayın**.
- Tüm alanları `private` yapın; dışarının erişmesi gerekiyorsa getter yazın.
- Her dosyanın başına ad, soyad, numara ve görev numarasını yorum olarak yazın.
- Kontrol sorularının yanıtlarını ilgili dosyanın sonuna yorum bloğu olarak ekleyin.

> **DİKKAT**
>
> İnternetten ya da yapay zekâ araçlarından alınan kodun **doğrudan** teslim edilmesi kabul edilmez. Yazdığınız her satırı açıklayabilmeniz beklenir; laboratuvar sorumlusu rastgele öğrencilerden kodunu açıklamasını isteyebilir. Özellikle `this(...)` zincirinizi neden o sırayla kurduğunuzu sorabilir.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre: 15 dakika · Puan: değerlendirmeye katılmaz, ancak zorunludur

Amaç

Geçen haftanın ortamının hâlâ çalıştığını doğrulamak ve bu haftanın çalışma klasörünü hazırlamak.

Görev Tanımı

1. Masaüstünde `OOP-Hafta02` klasörü oluşturun; içine `src` ve `bin` alt klasörlerini ekleyin.
2. VS Code'da **Dosya → Klasör Aç** ile bu klasörü açın.
3. `java -version` ve `javac -version` komutlarının sürüm bilgisi yazdırdığını doğrulayın.
4. `src` klasörüne `OnKosul.java` adında bir dosya oluşturun ve aşağıdaki kodu yazın. Derleyip çalıştırın.

OnKosul.java

```java
public class OnKosul {
    private String ad;

    public OnKosul(String ad) {
        this.ad = ad;
    }

    @Override
    public String toString() {
        return "OnKosul: " + ad;
    }

    public static void main(String[] args) {
        System.out.println(new OnKosul("hazir"));
    }
}
```

Kod 2.1 — Ön koşul denetim programı

▸ TERMINAL — Beklenen çıktı

```
$ javac -d bin src/OnKosul.java
$ java -cp bin OnKosul
OnKosul: hazir
```

> **SIK YAPILAN HATA**
>
> Bu program derlenmiyorsa sorun kodunuzda değil ortamınızdadır. `javac` tanınmıyorsa yalnızca JRE kurulu olabilir. Hafta 1 ders notundaki sorun giderme tablosuna bakın; çözemezseniz **hemen** laboratuvar sorumlusuna haber verin. Bu adım tamamlanmadan diğer görevlere geçilemez.

Kontrol Soruları

1. Yukarıdaki programda `OnKosul()` biçiminde parametresiz bir nesne üretebilir misiniz? Neden?
2. `this.ad = ad;` satırındaki `this` silinirse program derlenir mi? Çıktı ne olur?

Görev 1 — Kitap Sınıfına Tam Yapıcı

Süre: 20 dakika · Puan: 20 · Zorluk: ●○○

Amaç

Geçen hafta yazdığınız `Kitap` sınıfını, nesnenin doğduğu anda geçerli olmasını garanti eden bir tam yapıcıyla yeniden yazmak.

Görev Tanımı

`Kitap` adında bir sınıf yazın. Hafta 1'de bu sınıfın alanları `public` idi ve nesne üretildikten sonra tek tek doldurulyordu. Bu hafta alanları `private` yapacak ve tüm bilgileri alan bir tam yapıcı ekleyeceksiniz.

Gereksinimler

Öğe	Gereksinim
Alanlar	`private String baslik` · `private String yazar` · `private int sayfaSayisi` · `private int basimYili` · `private double fiyat`
Tam yapıcı	Beş parametrenin tamamını alır ve **doğrular**
Doğrulama — `baslik`	`null` veya boş ise `"Isimsiz"` kabul edilir
Doğrulama — `yazar`	`null` veya boş ise `"Bilinmiyor"` kabul edilir
Doğrulama — `sayfaSayisi`	Pozitif değilse `1` kabul edilir
Doğrulama — `basimYili`	1450–2100 aralığında değilse `2026` kabul edilir
Doğrulama — `fiyat`	Negatifse `0.0` kabul edilir
Getter metotları	Beş alan için de getter yazılır (setter **yazılmaz**)
`toString()`	`@Override` ile ezilir; biçimli tek satır üretir

Tablo 2.2 — Görev 1 gereksinimleri

> **İPUCU**
>
> Boş metin denetimi için `metin == null || metin.isBlank()` kullanın. `isBlank()` yalnızca boşluk karakterlerinden oluşan metinleri de yakalar; `isEmpty()` yakalamaz.

Sınıfı sınamak için `Gorev1_KitapDemo` adında ikinci bir sınıf yazın ve içinde en az iki kitap üretin.

```
▸ TERMINAL — Beklenen çıktı biçimi (ilk iki satır)
--- Kutuphane ---
Nesne Yonelimli Programlama  R. Benveniste     520 sf. 2024    480.00 TL [kalin]
Veri Yapilari                A. Demir          310 sf. 2019      0.00 TL
```

> **DİKKAT**
>
> Yapıcının önüne `void` **yazmayın**. Yazarsanız o artık bir yapıcı değil, `Kitap` adında sıradan bir metot olur ve `new Kitap(...)` satırınız `constructor Kitap in class Kitap cannot be applied to given types` hatası verir.

Kontrol Soruları

1. Alanları `private` yapmasaydınız hangi Hafta 1 sorunu geri gelirdi? Bir cümleyle açıklayın.
2. Doğrulama mantığını yapıcı yerine çağrı yerine (yani `main` içine) yazsaydınız hangi sorun ortaya çıkardı?
3. `toString()` metodunu yazmasaydınız `System.out.println(k1)` ne yazardı?

Görev 2 — Aşırı Yüklenmiş Yapıcılar ve this(...) Zinciri

Süre: 25 dakika · Puan: 20 · Zorluk: ●●○

Amaç

Aynı sınıf için birden çok üretim yolu sunmak ve gerçek işi tek bir yapıcıda toplayarak kod tekrarını yok etmek.

Görev Tanımı

Görev 1'deki `Kitap` sınıfına üç yapıcı daha ekleyin. **Doğrulama kodunu kopyalamayın**; her yeni yapıcı `this(...)` ile tam yapıcıya yönlensin.

Yapıcı imzası	Davranış
`Kitap(String baslik, String yazar, int sayfaSayisi, int basimYili)`	Fiyat bilinmiyor → `0.0` ile tam yapıcıyı çağırır
`Kitap(String baslik, String yazar)`	Yalnızca başlık ve yazar biliniyor → `1` sayfa, `2026` yıl, `0.0` fiyat
`Kitap(Kitap kaynak)`	**Kopya yapıcı** — kaynağın alan değerlerini taşıyarak bağımsız nesne üretir

Tablo 2.3 — Görev 2'de eklenecek üç yapıcı

Ayrıca aşağıdaki davranış metotlarını ve **denetimli** bir setter'ı ekleyin:

- `public void setFiyat(double fiyat)` — negatif değer verilirse ekrana uyarı yazar ve **değişiklik yapmaz**.
- `public boolean kalinMi()` — sayfa sayısı 400'den büyükse `true` döner.
- `public double sayfaBasinaFiyat()` — fiyatı sayfa sayısına böler.
- `public void indirimUygula(double yuzde)` — oran 0–100 aralığında değilse uyarı yazar ve değişiklik yapmaz.

- `public int yasi(int buguninYili)` — kitabın yaşını döndürür.

`Gorev1_KitapDemo` sınıfını genişletin: kopya yapıcının bağımsızlığını `==` ile sınayın ve bir `Kitap[]` dizisi üzerinde toplam değer ile en pahalı kitabı bulun.

```
▸ TERMINAL — Beklenen çıktı (son bölüm)
--- Kopya yapici testi ---
k1 == kopya -> false

Raf toplam degeri : 679.90 TL
En pahali kitap   : Nesne Yonelimli Programlama
```

> **SIK YAPILAN HATA**
>
> `this(...)` çağrısı yapıcının **ilk** deyimi olmak zorundadır. Önüne bir `System.out.println` satırı bile koyamazsınız; derleyici `call to this must be first statement in constructor` hatası verir.

> **İPUCU**
>
> Zincirin doğru kurulduğunu sınamak için tam yapıcıya geçici olarak bir `System.out.println("tam yapici calisti");` satırı ekleyin. Hangi yapıcıyı çağırırsanız çağırın bu satırın yazıldığını görmelisiniz. Sınamayı bitirince satırı **silin**.

Kontrol Soruları

1. Üç yapıcının hepsine doğrulama kodunu ayrı ayrı yazsaydınız, ileride “sayfa sayısı 10 000'den büyük olamaz” kuralını eklemek kaç yeri değiştirmeyi gerektirirdi?
2. `Kitap(String baslik, String yazar)` ile `Kitap(String yazar, String baslik)` aynı sınıfta bulunabilir mi? Neden?
3. `Kitap kopya = k1;` ile `Kitap kopya = new Kitap(k1);` arasındaki fark nedir? Hangisinde `k1 == kopya` sonucu `true` olur?

Görev 3 — this ile Alan Gölgeleme: Zaman Sınıfı

Süre: 25 dakika · Puan: 20 · Zorluk: ●●○

Amaç

Parametre adlarının alan adlarıyla aynı olduğu bir sınıf yazarak `this`'in neden zorunlu olduğunu bizzat deneyimlemek; `return this;` ile zincirleme çağrı kurmak.

Görev Tanımı

`Zaman` adında bir sınıf yazın. Sınıf gün içindeki bir anı saat, dakika ve saniye olarak tutar. **Tüm yapıcı parametrelerine alan adlarıyla aynı adı verin**; alanlara `this` ile ulaşın.

Yapıcı imzası	Davranış
`Zaman()`	Gece yarısı: `00:00:00`
`Zaman(int saat, int dakika)`	Saniye `0` kabul edilir
`Zaman(int saat, int dakika, int saniye)`	**Tam yapıcı.** Değerler 0–23 / 0–59 / 0–59 aralığına çekilir
`Zaman(int toplamSaniye)`	Gece yarısından itibaren geçen saniyeden zaman üretir
`Zaman(Zaman kaynak)`	Kopya yapıcı

Tablo 2.4 — Görev 3'te yazılacak beş yapıcı

Metotlar

Metot	Davranış
`public int toplamSaniye()`	Gece yarısından beri geçen saniye sayısı
`public Zaman ilerlet(int saniye)`	Verilen saniye kadar ilerletir; **`this` döndürür**; gün aşımında başa döner
`public int farkSaniye(Zaman digeri)`	İki zaman arasındaki mutlak fark
`public boolean ogledenOnceMi()`	Saat 12'den küçükse `true`
`toString()`	`SS:DD:SS` biçiminde, iki basamaklı (`%02d`)

Tablo 2.5 — Görev 3 metotları

`Gorev3_ZamanDemo` sınıfında sınıfı sınayın. Aşağıdaki dört deneyi mutlaka yapın:

1. Beş yapıcının her biriyle bir nesne üretip yazdırın.
2. `new Zaman(25, 70, 99)` üretip sınır denetiminin çalıştığını gösterin.
3. `new Zaman(23, 59, 30)` üzerinde `z.ilerlet(45).ilerlet(15);` zincirini çalıştırıp gün aşımını gözlemleyin.
4. Kopya yapıcı ile üretilen nesnenin bağımsız olduğunu `==` ile sınayın.

```
▸ TERMINAL — Beklenen çıktı (sınır ve zincir deneyleri)
```

```
--- Sinir denetimi (25:70:99) ---
new Zaman(25, 70, 99) -> 23:59:59

--- Zincirleme ilerletme ---
baslangic              : 23:59:30
45+15 sn sonra (gun asimi): 00:00:30
```

> **İPUCU**
>
> Gün aşımı için `%` (mod) işlecini kullanın: `(toplamSaniye() + saniye) % 86400`. Sonuç negatif çıkabileceği için `if (yeni < 0) yeni += 86400;` denetimini eklemeyi unutmayın.

> **DİKKAT**
>
> `ilerlet` metodunun parametresi de `saniye` adını taşır ve alanı gölgeler. Metodun içinde `this.saniye` yazmadan alana ulaşamazsınız. Bu, görevin **asıl öğretme noktasıdır**.

Kontrol Soruları

1. Tam yapıcıda `this` yazmayı unutursanız program derlenir mi? Çalışınca ne olur? Deneyin ve çıktıyı yazın.
2. `ilerlet` metodu `void` döndürseydi `z.ilerlet(45).ilerlet(15);` satırı derlenir miydi? Neden?
3. `Zaman(int toplamSaniye)` ile `Zaman(int saat, int dakika)` yapıcıları çakışır mı? `new Zaman(45296)` hangisini çağırır?

Görev 4 — Hata Ayıklama: Bozuk Yapıcılar

Süre: 20 dakika · Puan: 20 · Zorluk: ●●○

Amaç

Yapıcı ve `this` ile ilgili dört klasik hatayı tanımak, derleyici iletisini okuyup düzeltmek.

Görev Tanımı

Aşağıdaki kod **bilerek bozulmuştur**. İçinde dört ayrı hata vardır. Kodu `Gorev4_HataAyiklama.java` adıyla kaydedin, hataları bulun, her birinin **nedenini** kod içinde yorum olarak açıklayın ve düzeltin.

Gorev4_HataAyiklama.java (bozuk)

```java
public class Gorev4_HataAyiklama {
    public static void main(String[] args) {
        Urun u1 = new Urun("Klavye", 750.0, 3);
        Urun u2 = new Urun("Fare");
        Urun u3 = new Urun();
```

```
        System.out.println(u1);
    }
}

class Urun {
    private String ad;
    private double fiyat;
    private int stok;

    // HATA 1 ve 2 burada
    public void urun(String ad, double fiyat, int stok) {
        ad = ad;              // HATA 3
        fiyat = fiyat;
        stok = stok;
    }
    // HATA 4: eksik yapicilar

    @Override
    public String toString() {
        return String.format("%-10s %9.2f TL  stok: %3d", ad, fiyat, stok);
    }
}
```

Kod 2.2 — Bozuk kod — dört hata içerir

#	İpucu	Bakılacak bölüm
1	Yapıcının adı sınıf adıyla birebir aynı mı? Büyük/küçük harf dâhil.	Ders notu Bölüm 3.1
2	Yapıcının dönüş türü olabilir mi?	Ders notu Bölüm 6.1
3	Parametre adı alan adıyla aynıyken atama neyi neye yapıyor?	Ders notu Bölüm 4.1 ve 6.2
4	`new Urun("Fare")` ve `new Urun()` çağrıları hangi yapıcıyı arıyor?	Ders notu Bölüm 4.4 ve 6.3

Tablo 2.6 — Dört hata için ipuçları

Düzelttikten sonra sınıfa şu iki metodu da ekleyin ve sınayın:

- `public void setFiyat(double fiyat)` — negatif değer reddedilir.
- `public void stokEkle(int adet)` — stok negatife düşecekse işlem iptal edilir.

```
▸ TERMINAL — Düzeltildikten sonra beklenen çıktı (ilk üç satır)
Klavye        750.00 TL  stok:   3
Fare            0.00 TL  stok:   0
Isimsiz         0.00 TL  stok:   0
```

> **SIK YAPILAN HATA**
>
> Üçüncü hata **derleme hatası vermez**. Program derlenir, çalışır ve `null  0.00 TL  stok:  0` yazar. Bu, hatanın en tehlikeli türüdür: sessiz olan.

Kontrol Soruları

1. Dört hatadan hangileri derleme hatası, hangisi çalışma zamanında görünen mantık hatasıdır?
2. İkinci hatayı düzeltmeden `new Urun("Klavye", 750.0, 3)` çağrısı hangi derleyici iletisini verir? Birebir yazın.
3. Dördüncü hatayı `this(...)` zinciri kullanmadan da çözebilir miydiniz? Bunun sakıncası ne olurdu?

Görev 5 — İleri Düzey: Banka Hesabı (Bonus)

Süre: 25 dakika · Puan: 20 (bonus) · Zorluk: ●●●

Amaç

Yapıcı, `this`, kapsülleme ve kaynak yönetimini tek bir sınıfta birlikte kullanmak.

Görev Tanımı

`BankaHesabi` adında, `AutoCloseable` arayüzünü gerçekleyen bir sınıf yazın. Sınıf bir banka hesabını temsil eder; hesap kapandıktan sonra üzerinde işlem yapılamaz.

Öğe	Gereksinim
Alanlar	`private final String hesapNo` · `private final String sahip` · `private double bakiye` · `private int islemSayisi` · `private boolean acikMi`
Tam yapıcı	`(String hesapNo, String sahip, double baslangicBakiyesi)` — boş/`null` değerleri ve negatif bakiyeyi düzeltir, ekrana `[HESAP ACILDI]` yazar
Kısa yapıcı	`(String hesapNo, String sahip)` — `this(...)` ile tam yapıcıya `0.0` bakiyeyle yönlenir
`paraYatir(double)`	Pozitif olmayan tutarı reddeder, uyarı yazar
`paraCek(double)`	Geçersiz veya bakiyeden büyük tutarı reddeder
`transferEt(BankaHesabi hedef, double miktar)`	Bu hesaptan hedefe aktarım yapar; **her iki nesnenin de** `private` alanlarına erişir
`close()`	Hesabı kapatır, `[HESAP KAPANDI]` yazar; iki kez çağrılırsa ikincisi etkisizdir
Kapalı hesap denetimi	Kapalı hesapta işlem denenirse `IllegalStateException` fırlatılır
`toString()`	Hesap no, sahip, bakiye, işlem sayısı ve durum

Tablo 2.7 — Görev 5 gereksinimleri

`Gorev5_BankaDemo` sınıfında şu yedi adımı sırayla gösterin: hesap açma, işlemler, durum yazdırma, transfer, `try-with-resources` ile otomatik kapanış, kapalı hesapta işlem denemesi ve elle kapanış.

```
▸ TERMINAL — Beklenen çıktı (5. adım)
5) try-with-resources ile otomatik kapanis:
  [HESAP ACILDI] TR100003 - Gecici Hesap
  TR100003   Gecici Hesap     150.00 TL  islem: 1  acik
  [HESAP KAPANDI] TR100003 - son bakiye: 150.00 TL
```

BİLGİ

Aynı sınıfın **başka bir nesnesinin** `private` alanına erişebilirsiniz. `private`, nesne düzeyinde değil **sınıf düzeyinde** bir kısıttır. `transferEt` metodunda `hedef.bakiye += miktar;` yazabilmenizin nedeni budur. Bu, çoğu öğrencinin beklemediği bir kuraldır.

> **İPUCU**
>
> `final` alanlar yalnızca yapıcı içinde bir kez atanabilir. `hesapNo` ve `sahip` alanlarını `final` yaparsanız derleyici, onları sonradan değiştiren bir kod yazmanızı engeller.

Kontrol Soruları

1. `transferEt` metodunda `hedef.bakiye` yazabilmenizin nedeni nedir? `private` kısıtı hangi düzeyde çalışır?
2. `try-with-resources` bloğu içinde bir istisna fırlatılırsa `close()` yine çağrılır mı? Çıktıdaki sıralamayla gerekçelendirin.
3. `close()` metodunda `if (acikMi)` denetimi olmasaydı ne olurdu?

Değerlendirme ve Teslim

Puanlama Ölçütleri

Ölçüt	Puan	Açıklama
Yapıcı tasarımı	25	Tam yapıcı doğru; aşırı yükleme ve `this(...)` zinciri kurulmuş
Doğrulama ve kapsülleme	20	Alanlar `private`; kural yapıcı ve setter içinde, çağrı yerinde değil
`this` kullanımı	15	Alan gölgelemesi doğru çözülmüş; `return this;` yerinde kullanılmış
Doğru çalışma	10	Program derleniyor ve beklenen çıktıyı üretiyor
Kod okunabilirliği	5	Adlandırma kurallarına uygunluk, girinti, anlamlı yorumlar
Kontrol soruları	5	Sorular yorum olarak doğru yanıtlanmış
Görev 5 (bonus)	+20	İleri düzey görevin tamamlanması

Tablo 2.8 — Laboratuvar puanlama ölçütleri (Görev 1–4 toplam 80 puan)

Teslim Öncesi Kontrol Listesi

Öz Değerlendirme Kontrol Listesi

- [ ] Tüm dosyalar hatasız derleniyor (`javac -d bin src/*.java` hiçbir hata vermiyor).
- [ ] Her dosyanın başında ad, soyad, numara ve görev numarası yorumu var.
- [ ] Dosya adları içindeki `public` sınıf adlarıyla birebir aynı.
- [ ] **Hiçbir yapıcının önünde dönüş türü (`void` dâhil) yok.**
- [ ] Tüm alanlar `private`; dışarıdan doğrudan erişilen alan kalmadı.
- [ ] Parametre adı alanla aynı olan her yerde `this.` kullanılmış.
- [ ] Aşırı yüklenmiş yapıcılar `this(...)` ile tam yapıcıya zincirlenmiş; doğrulama kodu tekrarlanmamış.
- [ ] Her sınıfta `@Override public String toString()` var.
- [ ] Kopya yapıcının bağımsızlığı `==` ile sınanmış ve sonucu yazılmış.
- [ ] Kontrol sorularının yanıtları yorum olarak yazılmış.
- [ ] Tüm dosyalar `Hafta02_<Numara>_<AdSoyad>.zip` içinde toplanmış.

Sık Karşılaşılan Hatalar

Hata iletisi / belirti	Nedeni	Çözüm
`constructor Kitap in class Kitap cannot be applied to given types`	Verilen argümanlara uyan yapıcı yok; çoğu zaman varsayılan yapıcı kaybolmuştur	Uygun yapıcıyı yazın veya çağrıdaki argümanları düzeltin
`call to this must be first statement in constructor`	`this(...)` çağrısından önce başka bir deyim var	`this(...)` satırını yapıcının en başına taşıyın
`recursive constructor invocation`	Yapıcı zinciri döngüsel: A→B ve B→A	Zincirin ucunda gerçek işi yapan bir yapıcı bırakın
Alanlar `null` ve `0` kalıyor, hata yok	`this` unutulmuş; parametre kendine atanıyor	`this.alan = parametre;` biçiminde yazın
`fiyat has private access in Kitap`	`private` alana sınıf dışından erişilmeye çalışıldı	Getter kullanın veya işlemi sınıfın içine taşıyın
`variable hesapNo might already have been assigned`	`final` alana ikinci kez değer verilmiş	`final` alanı yalnızca yapıcıda bir kez atayın
`incompatible types: void cannot be converted to Zaman`	`ilerlet` metodu `this` yerine hiçbir şey döndürüyor	Dönüş türünü `Zaman` yapın ve `return this;` ekleyin

Tablo 2.9 — Laboratuvarda sık karşılaşılan sorunlar

SON SÖZ

Görevleri süre içinde tamamlayamazsanız panik yapmayın. Tamamladığınız kadarını teslim edin ve eksik kısmı bir sonraki laboratuvardan önce bitirin. Önemli olan hızlı kod yazmak değil, **ne yaptığınızı anlamaktır**. Görev 5 bonustur; Görev 1–4'ü sağlam bitirmek her zaman önceliklidir.